Programación orientada a objetos en C++: juego de naves en consola (class Nave)

Alexis Maier

E.E.S.T. N.º 1 «Eduardo Ader», Vicente López

Laboratorio de Programación (LPR) — 5.º año 3.ª división, grupos A-B

Profesor: Mansilla Muñoz York Elías

21 de octubre de 2026

v1.0.0

Programación orientada a objetos en C++: juego de naves en consola (class Nave)

Introducción

La programación orientada a objetos (POO) es la evolución natural del `struct`: una clase agrupa los datos (atributos) y también las funciones que los modifican (métodos) dentro de un mismo objeto. Su regla de oro es el encapsulamiento: las variables internas se declaran `private` y solo se accede a ellas mediante métodos `public`, que actúan como puertas controladas.

En esta actividad se desarrolló un simulador de combate espacial en consola con las clases `Nave`, `Asteroide` y `Bala`, se agregó la funcionalidad extra de **puntaje**, y se redactó la documentación técnica (informe APA v7, manual del programador y manual del usuario). Integrante: Alexis Maier (trabajo individual: código, pruebas, capturas, manuales e informe).

Desarrollo

Diseño de clases

`Nave` encapsula `x`, `y` y `vidas`, y ofrece `pintar()`, `borrar()`, `mover(char)`, `getVidas()` y `reducirVida()`. `Asteroide` y `Bala` encapsulan su posición y se mueven por sí mismas con `mover()`. En `main()` se crea `Nave jugador(35, 20, 3);`, se mantienen los asteroides y las balas en `vector<Asteroide*>` y `vector<Bala*>`, y el bucle principal lee el teclado (`_kbhit`, `_getch`), mueve los objetos y resuelve las colisiones.

Funcionalidad extra: puntaje

```
int puntaje = 0;                    // antes del bucle principal
...
gotoxy(30, 2); cout << "PUNTAJE: " << puntaje << " ";   // HUD
...
// en la colision Bala -> Asteroide:
puntaje += 10;
```

Compilación y evidencias

Se compiló con `g++ src/main.cpp -o src/juegodenaves.exe` sin errores. Las capturas se obtuvieron ejecutando el juego real en su propia consola de Windows, inyectándole teclas (nombre, movimiento y disparos) y volcando el contenido de la pantalla

de la consola. El HUD muestra el piloto «Alexis Maier», las vidas y el puntaje (en la Figura 1, 30 puntos).

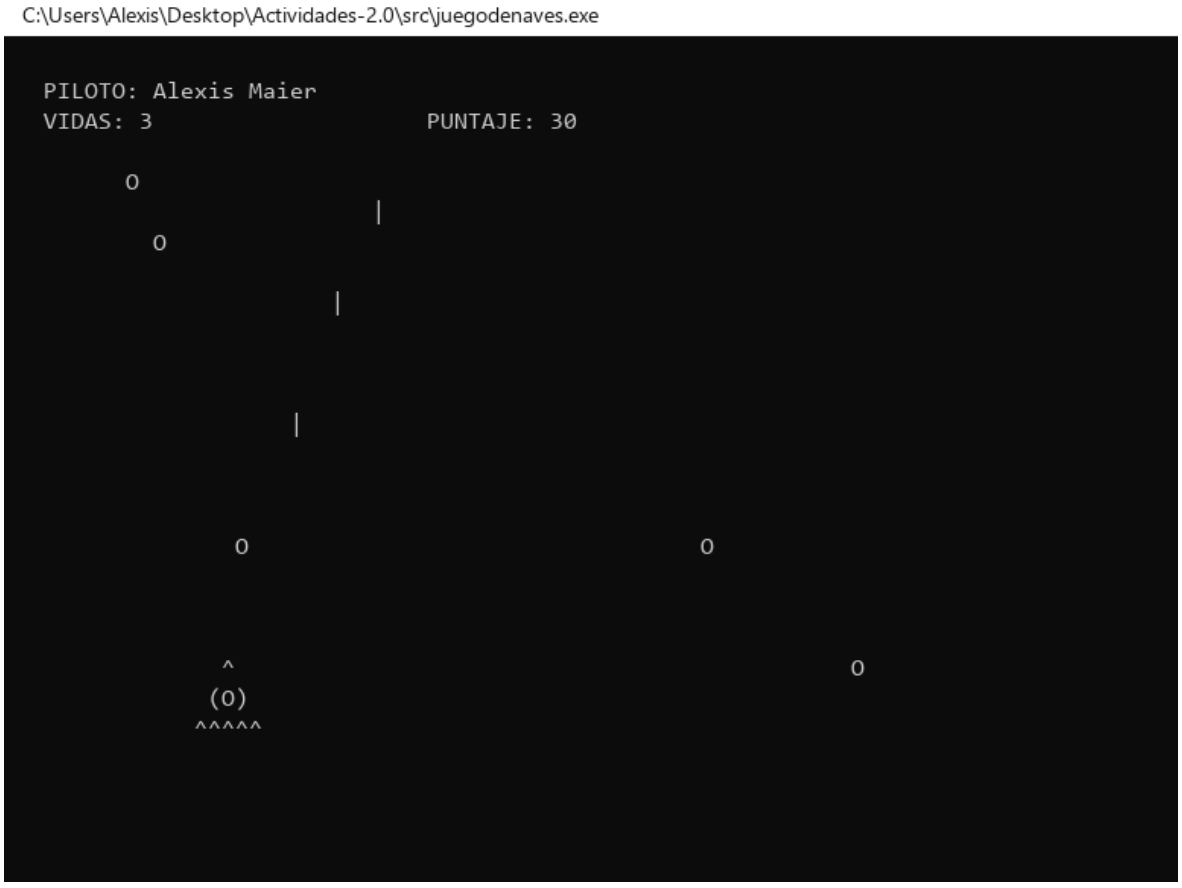


Figura 1. Combate en consola: HUD con nombre del piloto, vidas y puntaje, asteroides, balas y la nave.

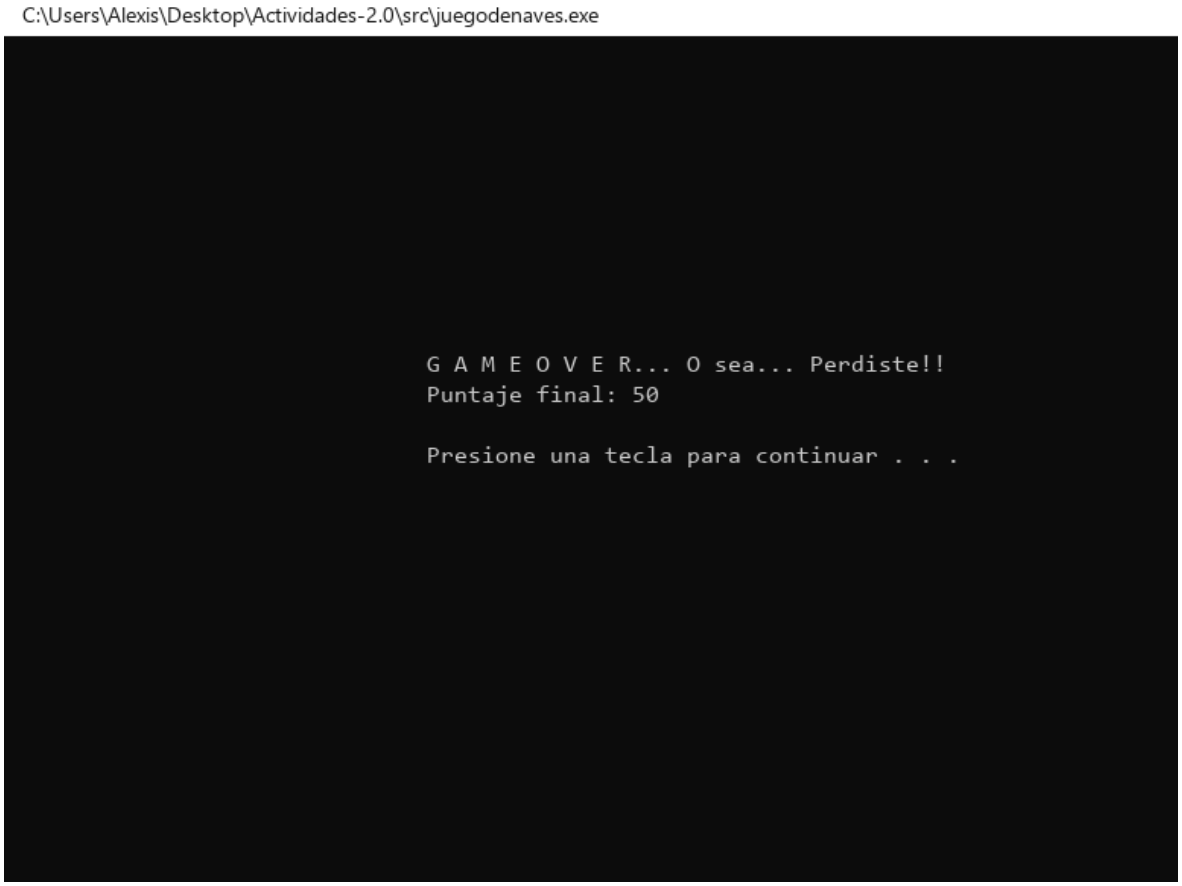


Figura 2. Pantalla final de Game Over tras perder las vidas, con el puntaje final.

Cuestionario técnico de control

1. struct vs class: ¿Cuál es la principal diferencia técnica entre un struct (visto en la Actividad 7) y una clase, especialmente respecto a la sección private y public?

En C++ la diferencia técnica es el acceso por defecto: los miembros de un `struct` son públicos y los de una `class` son privados. En la práctica, el `struct` se usa para agrupar datos, mientras que la clase, además de los datos (atributos), incluye funciones (métodos) y oculta sus variables en la sección `private:`, exponiendo solo métodos en `public:` como «puertas controladas». En la clase `Nave`, `x`, `y` y `vidas` son privados y `mover()` o `reducirVida()` son públicos.

2. Encapsulamiento: ¿Por qué es peligroso dejar la variable salud como pública? ¿Qué desastre podría causar un programador novato desde el main()?

Si `vidas` fuera pública, cualquier parte del programa podría modificarla sin control: un novato podría escribir `jugador.vidas = -50;` o `jugador.vidas = 9999;`, dejando al jugador con vidas negativas (el juego nunca terminaría porque la condición es `vidas == 0`)

o con invencibilidad. Con `private` el único camino es `reducirVida()`, que centraliza la regla y permite validarla en un solo lugar.

3. Constructor: Explicá con tus palabras para qué sirve el bloque Nave(string _nombre, ...) dentro del código.

El constructor es un método especial que se ejecuta automáticamente al crear el objeto y deja sus atributos en un estado inicial válido. En este código es `Nave(int _x, int _y, int _vidas)` (la guía menciona una versión con nombre `_nombre`, con la misma idea): al escribir `Nave jugador(35, 20, 3);` se asigna la posición inicial (35, 20) y las 3 vidas sin que `main()` tenga que tocar las variables privadas una por una.

4. Desafío funcionalidad: Explicá con tus palabras qué funcionalidad elegiste agregar de las 15 opciones y en qué partes del código tuviste que insertarla para que funcione.

Elegí agregar un **puntaje**: el jugador suma 10 puntos por cada asteroide destruido. Tuve que insertar tres cosas en `main()`: (1) la variable `int puntaje = 0;` antes del bucle principal; (2) la línea `puntaje += 10;` dentro de la colisión bala-asteroide, donde se destruye el asteroide; y (3) la impresión `gotoxy(30, 2); cout << "PUNTAJE: " << puntaje;` en el HUD, más el puntaje final en la pantalla de Game Over. La prueba automática de la Figura 1 muestra el puntaje subiendo de 10 en 10.

Conclusión

La clase `Nave` ejemplifica cómo la POO organiza el código: los datos quedan protegidos, las acciones viven junto a los datos y el constructor garantiza un estado inicial válido. La funcionalidad de puntaje se integró con tres cambios acotados y se corrigieron además fugas de memoria del código base (`delete` de los objetos reemplazados). Documentar el trabajo con informe, manuales y CHANGELOG forma parte del proceso profesional de desarrollo de software.

Referencias

cplusplus.com. (s. f.). *Classes (I)*. https://cplusplus.com/doc/tutorial/classes/

GeeksforGeeks. (s. f.). *Access modifiers in C++*.

https://www.geeksforgeeks.org/access-modifiers-in-c/

Microsoft. (s. f.). *Constructores (C++)*. https://learn.microsoft.com/es-es/cpp/cpp/constructors-cpp

Git. (s. f.). *Documentation*. https://git-scm.com/doc

Mansilla Muñoz, Y. E. (2026). *Actividad 8: Programación orientada a objetos (POO) en C++ — Juego de naves en consola* [Guía de actividad]. E.E.S.T. N.º 1 «Eduardo Ader».